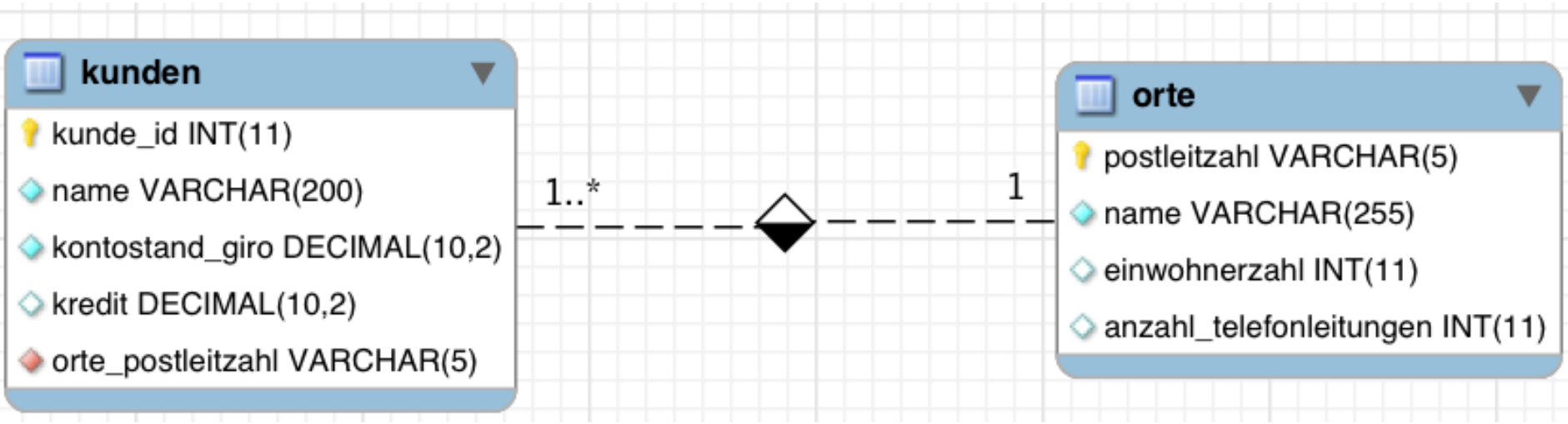
MySQL: Einfache Aggregatfunktionen

(= Funktionen, die etwas zusammenfassen)

Beispieldatenbank "Kunden"

kunden (kunde_id, name, ⇧orte_postleitzahl,
kontostand_giro, kredit)
orte (postleitzahl, name, einwohnerzahl,
anzahl_telefonleitungen)



Beispieldatenbank "Kunden"

kunden (kunde_id, name, ⇧ort_postleitzahl, kontostand_giro, kredit)
orte (postleitzahl, name, einwohnerzahl, anzahl_telefonleitungen)

postleitzahl	name	einwohnerzahl	anzahl_telefonleitungen
80995	München	1000000	385
79312	Emmendingen	40000	12
79111	Freiburg	280000	195
20095	Hamburg	2000000	1004

kunde_id	name	ort_postleitzahl	kontostand_giro	kredit
1	John	79111	182.00	-430320.22
2	Herbert	79312	10291.32	-10000.00
3	Sabina	79312	-253.21	-3205.32
4	Mary	79111	-832.01	NULL
5	Heinrich	79111	15302.85	0.00
6	Usal	80995	23012.21	NULL
7	Johannes	80995	159.31	0.00
8	Carla	79312	503.06	-15302.68
9	Ludowika	79111	25201.07	-82213.99
10	Niemand	99999	-5021.30	-3024.21

Aggregatfunktionen

Ähnlich wie in Excel:

- COUNT() → Datensätze zählen
- SUM() → Summe
- AVG() → Durchschnitt
- MAX() → Maximum
- MIN() → Minimum

gute Erklärung im Web:
http://www.teialehrbuch.de/Kostenlose-Kurse/SQL/14750-Aggregatfunktionen.html

MIN(), MAX()

```
SELECT
    *
FROM
    orte
```

MIN – gibt den kleinsten Wert einer Spalte zurück
MAX – gibt den höchsten Wert einer Spalte zurück

postleitzahl	name	einwohnerzahl	anzahl_telefonleitung...
80995	München	1000000	385
79312	Emmendingen	40000	12
79111	Freiburg	280000	195
20095	Hamburg	2000000	1004

MIN(), MAX()

```
SELECT
   MIN(anzahl_telefonleitungen)
FROM
   orte
```

postleitzahl	name	einwohnerzahl	anzahl_telefonleitung...
80995	München	1000000	385
79312	Emmendingen	40000	12
79111	Freiburg	280000	195
20095	Hamburg	2000000	1004

min(anzahl_telefonleitungen)
12

MIN(), MAX()

```
SELECT
   MIN(anzahl_telefonleitungen) AS min,
   MAX(anzahl_telefonleitungen) max (*)

FROM
   orte
```

postleitzahl	name	einwohnerzahl	anzahl_telefonleitung...
80995	München	1000000	385
79312	Emmendingen	40000	12
79111	Freiburg	280000	195
20095	Hamburg	2000000	1004

min	max
12	1004

(*) Erinnerung: AS kann weggelassen werden

MIN(), MAX()

Achtung: Fehler!

```
SELECT
   MIN(anzahl_telefonleitungen), name
FROM
   orte
```

postleitzahl	name	einwohnerzahl	anzahl_telefonleitung...
80995	München	1000000	[illegible]
79312	Emmendingen	40000	12
79111	Freiburg	280000	195
20095	Hamburg	2000000	1004

min(anzahl_telefonleitungen)	name
12	München

MIN/MAX fasst alle Datensätze zu EINEM Ergebnis zusammen (Aggregat).
Zusätzliche Auswahl eines Attributs zeigt hier den ersten Wert an ("München")

MIN(), MAX()

```
SELECT *, MIN(kontostand_giro)
FROM kunde
```

Achtung: Fehler!

Das geht nicht!

kunde_id	name	ort_postleitzahl	kontostand_giro	kredit	MIN(kontostand...
1	John	79111	182.00	-430320.22	-5021.30

Da Aggregierungs-Funktionen in WHERE-Clauses nicht erlaubt sind, ist eine Subquery nötig oder Kreativität:

```
SELECT *
FROM kunde
ORDER BY kontostand_giro ASC
LIMIT 1
```

COUNT(): Anzahl Datensätze

```
SELECT * FROM orte
```

postleitzahl	name	einwohnerzahl	anzahl
80995	München	1000000	385
79312	Emmendingen	40000	12
79111	Freiburg	280000	195
20095	Hamburg	2000000	1004

Wie viele Zeilen sind das?
(oder: Wie viele Datensätze bekommen wir als Ergebnis?)

COUNT(): Anzahl Datensätze

```
SELECT * FROM orte
```

postleitzahl	name	einwohnerzahl	anzahl
80995	München	1000000	385
79312	Emmendingen	40000	12
79111	Freiburg	280000	195
20095	Hamburg	2000000	1004

Wie viele Zeilen sind das?
(oder: Wie viele Datensätze bekommen wir als Ergebnis?)

```
SELECT
    COUNT(*) AS `Anzahl aller Orte`
FROM
    orte
```

Anzahl aller Orte
4

COUNT(): Anzahl Datensätze

```
SELECT * FROM orte
```

postleitzahl	name	einwohnerzahl	anzahl
80995	München	1000000	385
79312	Emmendingen	40000	12
79111	Freiburg	280000	195
20095	Hamburg	2000000	1004

COUNT() gibt die Anzahl der Ergebnis-Datensätze zurück

`SELECT COUNT(*) FROM xy` gibt also die Anzahl der Ergebniszeilen von `SELECT * FROM xy` zurück.

```
SELECT
    COUNT(*) AS `Anzahl aller Orte`

FROM
    orte
```

Anzahl aller Orte
4

COUNT(): Anzahl Datensätze

```
SELECT
    *
FROM
    kunden;
```

kunde_id	name	ort_postleitz...	kontostand_giro	kredit
1	John	79111	182.00	-430320.22
2	Herbert	79312	10291.32	-10000.00
3	Sabina	79312	-253.21	-3205.32
4	Mary	79111	-832.01	NULL
5	Heinrich	79111	15302.85	0.00
6	Usal	80995	23012.21	NULL
7	Johannes	80995	159.31	0.00
8	Carla	79312	503.06	-15302.68
9	Ludowika	79111	25201.07	-82213.99
10	Niemand	99999	-5021.30	-3024.21

```
SELECT
    COUNT(*) `Anzahl aller Kunden`
FROM
    kunden;
```

Anzahl aller Kunden
10

COUNT(): Anzahl Datensätze

```
SELECT
    kredit
FROM
    kunden;
```

kredit
-430320.22
-10000.00
-3205.32
NULL
0.00
NULL
0.00
-15302.68
-82213.99
-3024.21

COUNT(): Anzahl Datensätze

```
SELECT
    kredit
FROM
    kunden;
```

kredit
-430320.22
-10000.00
-3205.32
NULL
0.00
NULL
0.00
-15302.68
-82213.99
-3024.21

Was ergibt die Abfrage

```
SELECT
    COUNT(kredit)
FROM
    kunden;
```

?

COUNT(): Anzahl Datensätze

```
SELECT
    kredit
FROM
    kunden;
```

kredit
-430320.22
-10000.00
-3205.32
NULL
0.00
NULL
0.00
-15302.68
-82213.99
-3024.21

10 Ergebniszeilen
(2 davon enthalten keinen Wert – NULL)

COUNT(): Anzahl Datensätze

```
SELECT
    COUNT(kredit)
FROM
    kunden;
```

COUNT(kredit)
8

kredit
-430320.22
-10000.00
-3205.32
NULL
0.00
NULL
0.00
-15302.68
-82213.99
-3024.21

COUNT(): Anzahl Datensätze

```
SELECT
    COUNT(kredit)
FROM
    kunden;
```

ergibt 8
(alle Datensätze, die einen Wert haben).

kredit
-430320.22
-10000.00
-3205.32
NULL
0.00
NULL
0.00
-15302.68
-82213.99
-3024.21

Wir wollen aber die Kunden, deren Kredit 0.00 ist, nicht mitzählen.
Erweitern Sie die Abfrage entsprechend.

COUNT(): Anzahl Datensätze

```
SELECT
    COUNT(kredit)
FROM
    kunden
 WHERE
    kredit < 0; -- nur negative Werte

...
 WHERE
    kredit != 0; -- ungleich 0

...
 WHERE
    kredit < 0 OR kredit > 0;
      -- ungleich 0, wie oben
```

kredit
-430320.22
-10000.00
-3205.32
NULL
0.00
NULL
0.00
-15302.68
-82213.99
-3024.21

COUNT(kredit)
6